HW7 – Design D2

Team Members:

Noah Schumacher

Ryan Broyles

Manjinder Kaur

Minh Nguyen

Akshat Chaturvedi



1. System Introduction:

SkillForge is a tool envisioned to provide a user-focused web interface and research-backed technology to support robotics training from human demonstrations. As part of being user-focused, SkillForge aims to provide robotics training using data and tooling common industrial applications would have access to, rather than focus on a narrow band on high-specific constraints used in academic research to maximize benchmark performance.

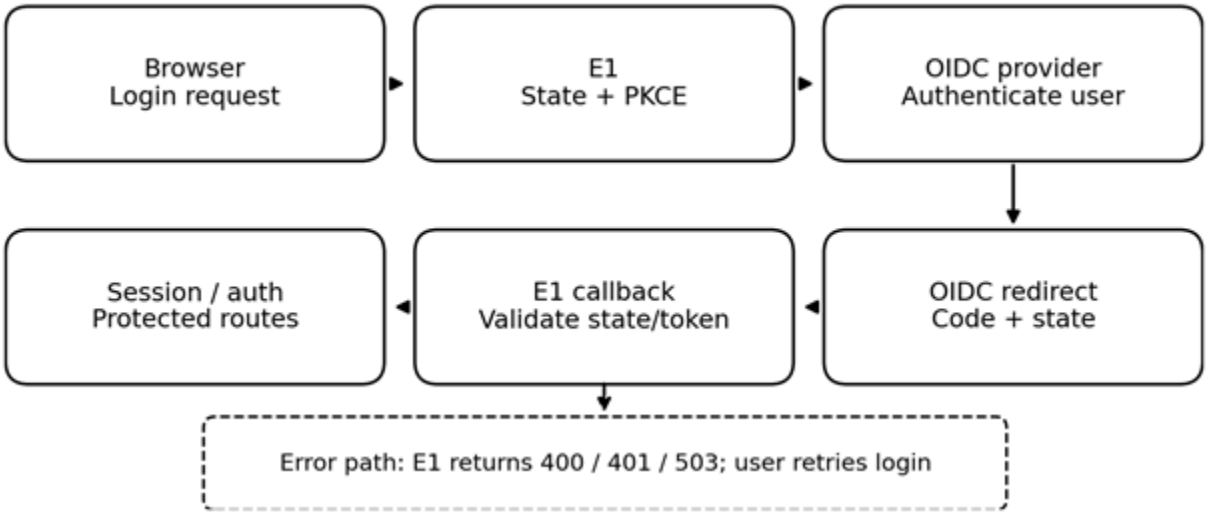
To minimize unwieldy constraints on the system requirements, SkillForge seeks to train robots using commonly available 3rd-person perspective video in a simulation environment. This allows for capturing tasks for training using basic camera systems and web-scaped data and removes the necessity for users to know how to setup custom real-world robot training environments. Additionally, a web interface will be built to support user customization and interaction. While academic research has developed individual components of this system, no work is yet to combine all components together into a unified product.

The design of SkillForge is broken into the frontend (C1-C4) and backend (E1-E4) components with API communication and database stores managing communication and context. In design D1, our team designed components E2-E4 (most of which is also included in this document for clarity), while D2 focuses on E1 and C1-C4.

2. D2 Diagrams – Data Flow, Core Algorithms, Build vs Reuse, API Contract

E1

Data Flow:



Data model: No persistent SkillForge identity data. The external OIDC provider owns user accounts; E1 keeps only short-lived login state and an ephemeral authenticated session. Do not store user passwords or long-lived tokens in E1.

D0 alignment: E1 performs I11 (login request) and I12 (login response). The proposed OIDC redirect/callback flow refines D0's generic credential exchange without bringing passwords through SkillForge.

Core Algorithms:

Algorithm 1: OIDC Authorization and Login

What	**Authenticate a user through a hosted OpenID Connect provider using authorization-code flow with PKCE.**
I & O	Input: browser login request; return path (optional String, allowlisted); callback code (required String); state (required String). Output: 302 redirect to hosted provider, then 302 to app with Secure/HttpOnly/SameSite session cookie; no password enters SkillForge.
Complexity	O(1) application bookkeeping per login (bounded login state). External provider latency dominates. At a proposed 10 concurrent logins, and again at 1,000 (100x), require nonblocking network I/O or a larger worker pool.
Why	Reuses mature identity management instead of building password hashing, reset flows, and MFA. Alternative: custom credential store (rejected due to security burden).

Edge Cases	User cancels, missing code, mismatched/expired state (<=10 min), replayed callback, unknown redirect, OIDC outage. Fail closed; show a retryable login error without leaking tokens.

Algorithm 2: Authentication and Session Validation

What	**For each protected call, validate the active session and obtain the stable provider user ID.**
I & O	Input: Secure HttpOnly session cookie (required for protected routes). Output: authenticated Boolean, user_id String, or HTTP 401. Session expiry is an absolute timestamp in UTC.
Complexity	O(1) lookup for a local ephemeral session; where token signature verification is used, O(T) to parse a token of T bytes. Propose ~10 active users initially and ~1,000 at 100x.
Why	Centralized validation prevents E2/E4 from implementing separate login systems. Reject invalid/expired sessions by default.
Edge Cases	Missing/expired cookie, restart clears in-memory sessions, concurrent logout and request, user account disabled, time skew. Return 401; browser can reauthenticate.

Build vs Reuse Decisions

What	**B or R**	**Library / Service**	**License**	**Reason**
Hosted authentication	Reuse	OIDC provider (Auth0 candidate)	Commercial service terms	Mature hosted login, account recovery and optional MFA; low prototype maintenance. Vendor and pricing need team approval.
OIDC client integration	Reuse	Authlib for Python	BSD-3-Clause	Standards-based OAuth/OIDC integration without custom cryptographic protocol code.
Authentication routes	Build	FastAPI auth adapter (proposed)	Custom code; FastAPI MIT	Implements redirects, allowlisted return targets, session checks and application-specific error messages.

Session management	Build + reuse	Short-lived server-side session + secure cookie	Custom code / standard browser cookie	Avoids storing user passwords; requires server session cleanup and reauthentication after restart.

Decision status: Proposed design. E1 vendor and session implementation must be approved by the group; D0 defines responsibilities but does not select an authentication platform.

API Contract

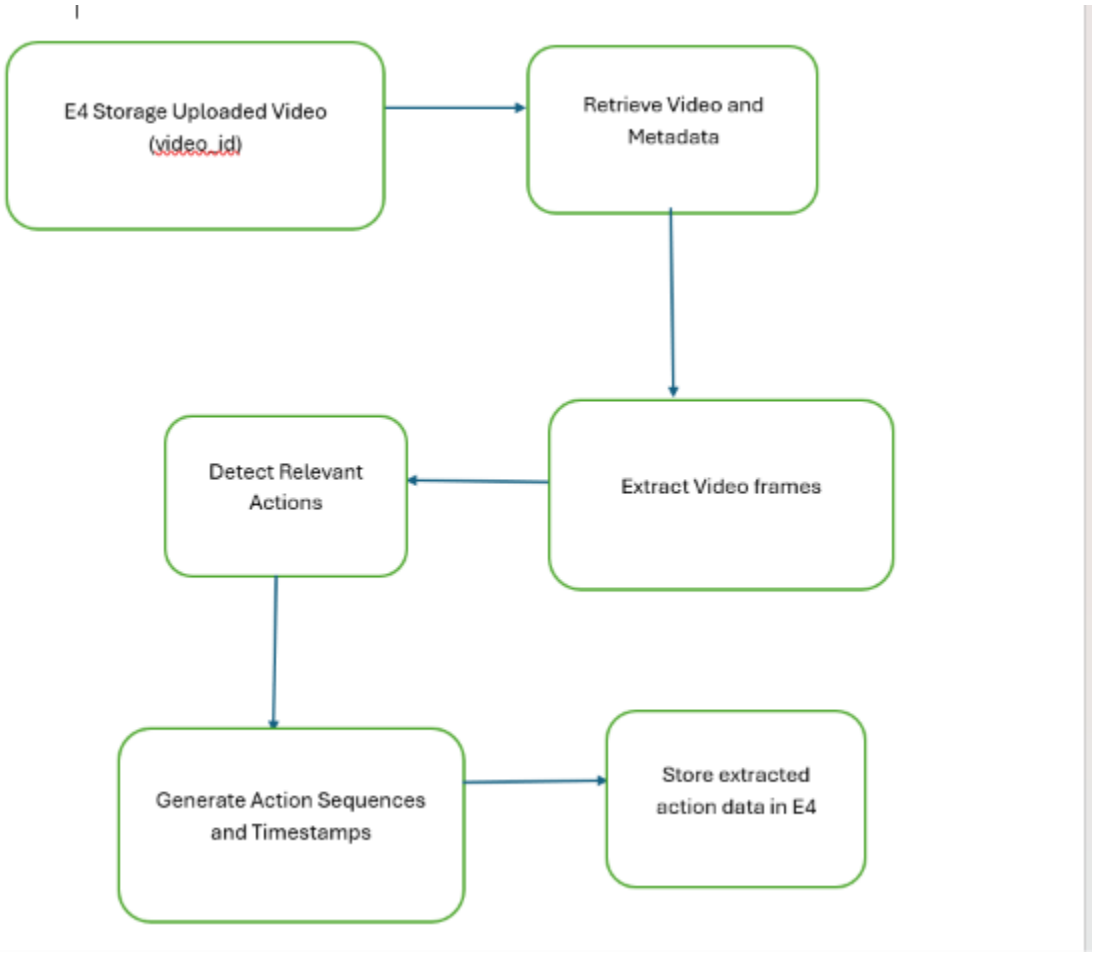
Endpoint / Method	**Inputs**	**Outputs**	**Errors**
GET /api/v1/auth/login	return_to: optional String, local app path, <=256 chars; defaults to /	302: Location=OIDC authorization URL	400 unapproved return_to; 503 provider configuration unavailable
GET /api/v1/auth/callback	code: required String <=2048 chars; state: required String <=256 chars	302 to return path; Set-Cookie for opaque authenticated session	400 missing/mismatched/expired state; 401 invalid auth response; 503 provider unavailable
GET /api/v1/auth/me	Authenticated session cookie: required	200: {authenticated: Boolean, user_id: String}	401 no valid session; 503 session service unavailable
POST /api/v1/auth/logout	Session cookie: optional	204 No Content; clear session cookie	500 session revocation failure, if session exists

Example: GET /api/v1/auth/me with an active session cookie -> 200 {"authenticated":true,"user_id":"oidc|example-123"}. Without a valid session -> 401 {"error":{"code":"UNAUTHORIZED","message":"Sign in required."}}.

Versioning: Public SkillForge routes use /api/v1; removing or renaming required fields, changing identity semantics, or changing authentication requirements is breaking and requires /api/v2. OIDC identity protocols remain provider-standard.

E2

Data Flow:



Data model: ActionSegment {segment_id: Integer, video_id: Integer (FK), start_s: Float, end_s: Float, action_label: String, confidence: Float 0..1, metadata: JSON}; ExtractionResult {video_id: Integer, status: String, segments: ActionSegment[], artifact_key: String|null}. Segment timestamps are seconds from the beginning of the video. Proposed schema; E4 persistence needs team approval.

Core Algorithms:

Algorithm 1: Video Decoding and Temporal Sampling

What: Decode third-person demonstration video and sample ordered frames for downstream motion analysis.

I&O: Input video_id (Integer >=1), sampling_fps (Float >0), VideoAsset reference from E4. Output timestamped RGB frames and source metadata; no original video modification.

Complexity: O(F) frame visits for F decoded frames; bounded streaming memory O(B) for a fixed frame batch. At 100x video duration, decoding work grows approximately 100x.

Why: Reuse established decoding instead of implementing codecs. Sampling reduces processing cost while preserving temporal ordering.

Edge Cases: Missing video, unsupported codec, zero-duration clip, corrupted frames, variable frame rate, and sampling_fps greater than source FPS. Reject invalid files or record skipped frames.

Algorithm 2: Action Segmentation and Extraction

What: Detect meaningful changes in motion across sampled frames and group consecutive observations into time-bounded candidate actions.

I&O: Input timestamped frames and optional reconstructed scene/motion features. Output ActionSegment[] with start_s, end_s, label, confidence, and metadata; uncertain actions receive an unknown label for review.

Complexity: O(F × M) for F sampled frames and per-frame model inference cost M; temporal grouping O(F). Batch size limits peak memory. A 100x increase in frames requires approximately 100x inference work for fixed model size.

Why: Separate extraction from E3 training so actions can be inspected, reused, and retried without retraining. A fixed threshold-only approach is simpler but brittle for varied viewpoints.

Edge Cases: No visible action, camera shake, occlusion, multiple actors, overlapping actions, ambiguous labels, duplicate requests, and model timeout. Preserve low-confidence results rather than asserting certainty.

Build vs Reuse Decisions:

Reuse | FFmpeg/PyAV (FFmpeg licensing depends on build; PyAV BSD) | Robust decoding, timestamps, and frame sampling.

Reuse | OpenCV (Apache 2.0) | Frame transforms and basic motion feature processing.

Build | Python extraction orchestration (custom code) | Video-specific sampling, segmentation, confidence thresholds, and structured output.

Reuse | Existing research-backed motion/action models (license to verify) | Avoid training a foundational perception model from scratch; model selection remains open.

Build + reuse | FastAPI and E4 metadata/storage APIs (FastAPI MIT; custom integration) | Validate requests and persist artifacts without duplicating file storage.

API Contract (proposed /api/v1; coordinate final ownership with E4):

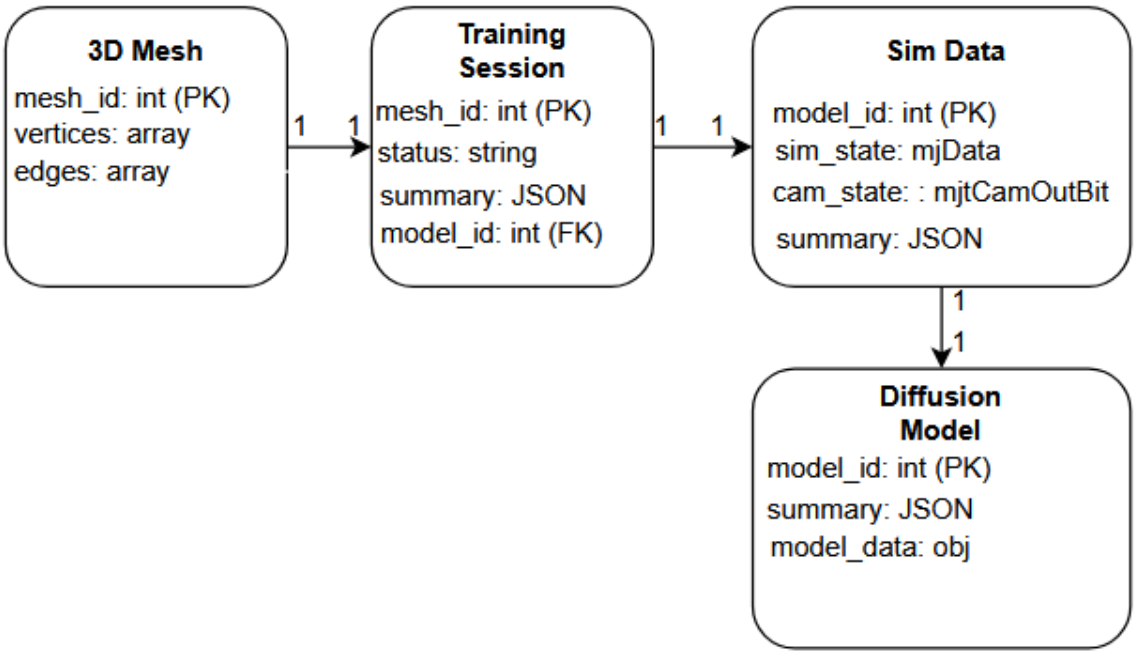
POST /api/v1/action-extractions | JSON {video_id: Integer >=1, sampling_fps: Float >0, options?: Object} | 202 {extraction_id: Integer, video_id: Integer, status: "queued"} | 404 missing video; 422 invalid settings; 409 duplicate active job; 500 internal error.

GET /api/v1/action-extractions/{extraction_id} | path Integer >=1 | 200 {extraction_id: Integer, video_id: Integer, status: String, segments: Array, artifact_key: String|null} | 400 invalid ID; 404 not found; 500 processing/storage failure.

Example: POST {"video_id":12,"sampling_fps":5.0} → 202 {"extraction_id":7,"video_id":12,"status":"queued"}. Status progresses queued → running → completed/failed. The API returns metadata; large binary intermediates remain in E4 storage. Breaking changes require /api/v2.

E3

Data Flow:



Core Algorithms:

Algorithm 1: Physics Model Training

What	Simulation Engine (MuJoCo) – Data from the simulations is used to train the physics model.
I & O	Inputs: 3D Mesh File Outputs: Live simulation state (mjData), Live camera state (mjtCamOutBit), Summary(JSON), Status(String)
Complexity	Highly variable – depends on complexity of 3D mesh surfaces and scales with the number of objects and frequency of collision.
Why	MuJoCo was designed with robotics policy training in mind and is an incredibly advanced physics engine that is open source. Alternative option is Isaac Sim, however due to budget constraints we are not able to acquire the appropriate graphic cards needed to utilize Isaac Sim so it was dropped in favor of MuJoCo.
Edge Cases	It is possible for the input to be corrupted if video processing fails, so it is important for checks to be done to make sure the data is 'clean' before training the model, especially since training is a lengthy process. Empty input is automatically returned with an error to component E2. Duplicate

	inputs will need a check, either against current summaries stored in the backend, or checking status, to find other physics model(s) that have already been trained on the input.

Algorithm 2: Diffusion Model Training

What	Diffusion Model – Trained from prepared robotic actions developed from the physics model.
I & O	Inputs: Robotic Action Sequences Output: Diffusion Model
Complexity	Highly variable – Training process can greatly different depending on the size and scale of the processed action
Why	Creating our own diffusion model allows us to train it and scale to our own specifications and embed it into our own systems more easily.
Edge Cases	Action Sequences are too large, non-existant.

Build vs Reuse Decisions

What	B or R	Library/Serv	License	Reason
Physics Engine	Reuse	MuJoCo	Apache 2.0	Provides a physics engine that is able to read our produced mesh files easily. Creating our own physics engine is way out of scope.
Diffusion Model	Build	None	None	Diffusion model will be built instead of using a pre-existing one to allow us to tailor it more to the results from our physics engine and control the training process

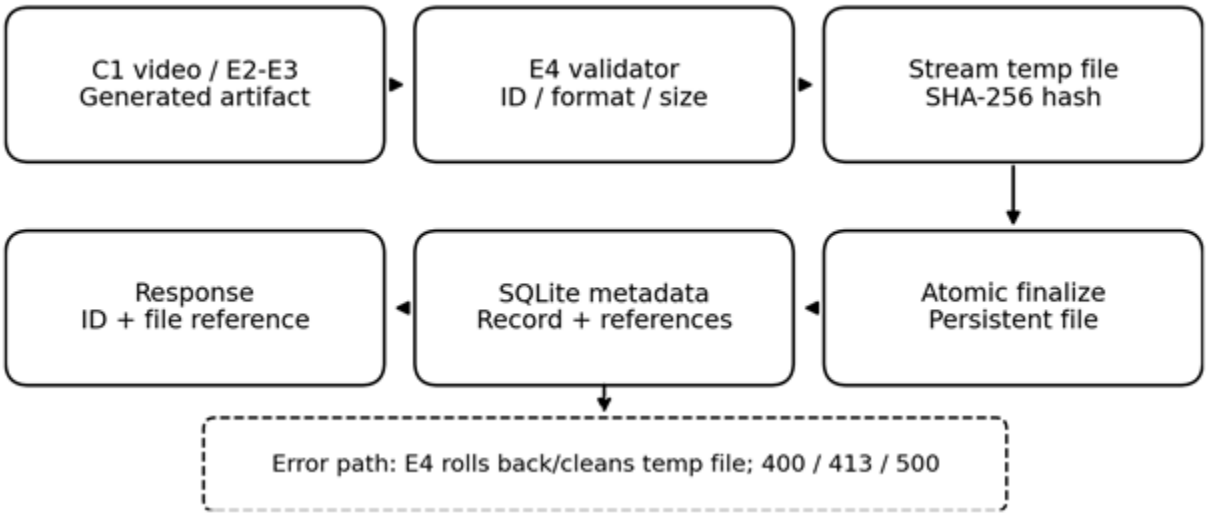
API Contract

Proposed REST API: /api/v1.

Endpoints	Inputs	Outputs	Errors
GET /api/v1/phys_model/{model_id}	model_id: path int >= 1. required	200: {summary: JSON, status: string}	400: invalid model id. 404: model not found
POST /api/v1/phys_model	muj_mjcf : MuJoCo XML. required	201: {model_id: int, summary: string}	500: Database error
GET /api/v1/diffusion_model/{model_id}	model_id image dataset training parameters	200: {summary: JSON, status: string}	400: invalid model id.
POST /api/v1/diffusion_model	model_id	201: {model_id: int, summary: string}	500: Training error

E4

Data Flow:



Data model (from D1): VideoAsset 1-to-many TrainingJob and PointCloudCache; TrainingJob 1-to-many ModelArtifact. Each record has a positive integer primary key. TrainingJob stores

training parameters and status; file records store location, format, size, and checksum. Large files live on disk, not in database BLOBs.

Interface alignment: D0 I15/I16 describe E2 <-> E4 FTPS model transfer, but D2 C1 uses HTTP /api/v1/videos. This draft proposes a consistent REST metadata/upload interface; reconcile I15/I16 file-transfer responsibility with E2 before final submission.

Core Algorithms:

Algorithm 1: File Storage and Retrieval

What	**Persist video demonstrations, point clouds and model artifacts; retrieve them by unique record ID.**
I & O	Input: binary file stream; kind enum {video, point_cloud, model}; associated positive Integer ID; metadata JSON. Output: ID Integer, storage_key String, size_bytes Integer (bytes), sha256 String (64 hex chars).
Complexity	O(S) transfer and checksum time for file size S; O(B) auxiliary memory with streaming chunk size B = 4 MiB. Example workload: 20 x 200 MB video files = ~4 GB; 100x = ~400 GB before point clouds/models.
Why	SQLite metadata + filesystem avoids huge relational BLOBs and suits a single-server research prototype. Alternative: store all binaries in SQLite (higher DB maintenance overhead).
Edge Cases	Empty file, unsupported MP4/MOV container, >2 GiB upload, duplicate hash, failed transfer, disk full, orphan temporary file, missing or corrupt stored artifact. Reject or clean temporary file before returning error.

Algorithm 2: Training Metadata and Status Management

What	**Create a training job, record parameter settings, update progress and associate stored outputs with their originating video and job.**
I & O	Input: video_id Integer >=1; parameters JSON object; status enum {queued,running,completed,failed}; progress_pct Float 0-100. Output: training_id Integer, status String, progress_pct Float, artifact references list of Strings.
Complexity	O(log N) per indexed ID lookup/update for N job records (database implementation dependent). Proposed initial N=20 and 100x N=2,000; both are small enough for indexed relational queries.

Why	Foreign keys and transactions maintain links among VideoAsset, TrainingJob and ModelArtifact. Alternative: loose per-job JSON files lack built-in integrity checks.
Edge Cases	Nonexistent video, invalid parameters, duplicate training submission, completion update before start, concurrent updates, training failure, artifact record created before file exists. Enforce transitions and atomic updates.

Build vs Reuse Decisions

What	B or R	Library / Service	License	Reason
Database engine	Reuse	SQLite	Public domain	Embedded and stable; efficient at small scale without a separate server. PostgreSQL is a scaling alternative.
Schema and constraints	Build	Custom relational schema	Custom code	Design-specific foreign keys and indexes on video_id, training_id and status.
Database access	Reuse	Python sqlite3	Python PSF License	Mature bundled driver with transactions; avoids an extra dependency.
File upload / retrieval	Build	Python streaming + filesystem	Custom code	Store large binaries externally, with checksums and atomic metadata writes.
Hash verification	Reuse	Python hashlib / SHA-256	Python PSF License	Standard library and robust content-integrity checking.
Video metadata / compression	Reuse	FFprobe/FFmpeg (optional)	LGPL/GPL per build	Mature video utilities; optional H.264 copies, preserve original for reconstruction quality.
API framework	Reuse	FastAPI (proposed)	MIT	Typed validation and OpenAPI docs; team needs to confirm deployment/skills.

Decision status: SQLite, local disk, and FastAPI remain proposed D1-derived choices. Determine expected hosting scale, storage quotas, and concurrent uploads before treating them as final.

API Contract

Endpoint / Method	Inputs	Outputs	Errors
POST /api/v1/videos	file: required binary MP4/MOV, 1 byte-2 GiB	201: {video_id: Int, storage_key: String, size_bytes: Int, sha256: String}	400 empty/unsupported; 409 duplicate hash if enabled; 413 oversized; 500 storage unavailable
GET /api/v1/videos/{video_id}	video_id: required Integer >=1	200: {video_id: Int, storage_key: String, size_bytes: Int, duration_s: Float|null}	400 invalid ID; 404 not found; 500 DB failure
POST /api/v1/training-jobs	JSON: video_id required Int >=1; parameters required object	201: {training_id: Int, video_id: Int, status: "queued", progress_pct: Float=0}	404 video not found; 422 invalid parameters; 500 DB failure
GET /api/v1/training-jobs/{id}	id: required Int >=1	200: {training_id: Int, video_id: Int,	400 invalid ID; 404 missing; 500 DB failure

		parameters: Object, status: String, progress_pct: Float}	
PATCH /api/v1/training-jobs/{id}	id: Int >=1; JSON status required enum {queued,running,completed,failed}; progress_pct optional Float 0..100	200: {training_id: Int, status: String, progress_pct: Float}	404 missing; 409 illegal transition; 422 invalid field
POST /api/v1/training-jobs/{id}/artifacts	id: required Int >=1; file: required nonempty binary; model_type: required String 1..64	201: {model_id: Int, training_id: Int, storage_key: String}	404 job missing; 413 too large; 500 disk failure
POST /api/v1/videos/{id}/point-clouds	id: required Int >=1; file: required nonempty PLY/NPZ binary	201: {cache_id: Int, video_id: Int, storage_key: String}	400 invalid format; 404 video missing; 500 disk failure

Example request: POST /api/v1/training-jobs, Content-Type: application/json

{"video_id":12,"parameters":{"epochs":100,"learning_rate":0.001,"seed":42}}

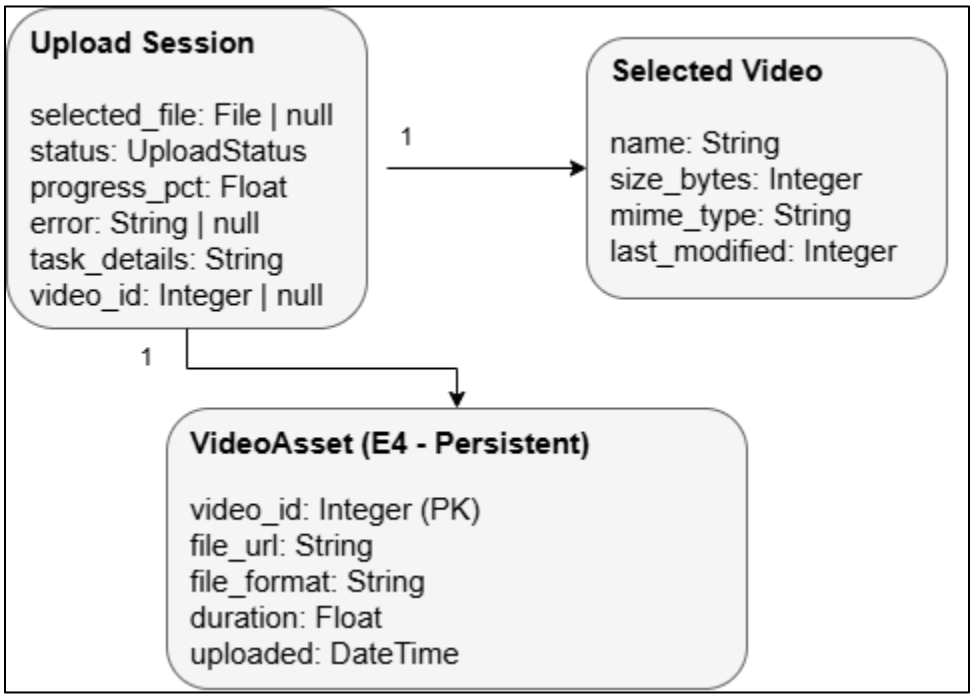
Example response (201): {"training_id":35,"video_id":12,"status":"queued","progress_pct":0.0}

Example error (404): {"error":{"code":"VIDEO_NOT_FOUND","message":"Video ID 12 does not exist."}}

Versioning: Paths use /api/v1. Removing/renaming required fields, changing units or changing field semantics requires /api/v2. JSON is for metadata; large files use multipart uploads or streaming downloads.

C1

Data Flow:



Structural decisions

Structural Decision	**Decision Rationale**
UploadSession = Frontend state	Tracks file selection, validation, transfer status, and errors without requiring database operations.
SelectedVideo = Browser File metadata	Allows users to inspect the selected video before uploading.
VideoAsset = E4 entity	Uses the existing database entity and storage system defined in D1.
UploadStatus = Enum	Proposed values: idle, validating, uploading, completed, failed, cancelled. Prevents ambiguous UI states.
video_id = Integer	Matches E4's primary key and existing REST API contract.
C1 → E4 = Temporary-to-persistent relationship	One upload session may create one successful video record. An upload session itself is not permanently stored.
Browser file content = Temporary	Avoids maintaining additional in-browser copies of large binary files.

Upload progress = Float 0–100	Enables the interface to display bytes transmitted as a percentage of total request bytes.
Task details = String	Records the user's description temporarily, pending a backend schema decision on persistence.
Data model = Client state + E4 relational reference	Avoids creating duplicate storage or database structures in the frontend.

Core Algorithms:

Algorithm 1: Video Selection and Client-Side Validation

What	Allow the user to select a local video file, extract basic metadata, and validate file type and size before initiating an upload.
I&O	**Input**: Browser File object; optional task description (String). **Output**: Validation result (Boolean), file metadata (name, size_bytes, mime_type), and error messages (String array)
Complexity	O(1) time and auxiliary memory for validation against fixed metadata constraints. Reading a file header for additional format verification requires O(H) time and memory for H header bytes, which is effectively constant for a bounded header.
Why	Prevents invalid submissions before expensive file transmission and allows the user to correct errors immediately. Browser validation is preliminary; backend validation remains authoritative.
Edge Cases	No file selected, empty file, incorrect extension, unsupported MIME type, file larger than 2 GiB, extension/MIME mismatch, or an inaccessible local file. Files using supported containers but unsupported video codecs may pass frontend checks and fail backend validation.

Algorithm 2: Video Download Speed and Transfer Speed to Database

What	Video Streaming: Algorithm used to download user-uploaded video. We will utilize the JavaScript Fetch API combined with a

	ReadableStream reader to process and save chunks without loading the entire file into RAM.
I&O	Input: User video file location Output: Video is fully downloaded and stored
Complexity	Algorithmic complexity is formally O(N) for time and space, but more relevant are elements such as download connection speed and client-server locations.
Why	Overly slow data download increases user frustration and degrades user experience at the very beginning of use. Using streaming downloads allows for cleaner video processing and allows initial ML pipeline work to begin while the server is still working on the download.
Edge Cases	File is too large to upload; user cancels upload mid-process; Internet dies on either end during upload process; part of file in corrupted but not detected prior to upload beginning.

Build vs Reuse Decisions

What	**B or R**	**Library / Service**	**License**	**Reason**
Video file selection	Reuse	Native HTML File API	Browser standard	Supports local file selection and metadata inspection without requiring a third-party library.
File validation	Build	TypeScript functions	Custom work	Implements SkillForge-specific file extension, MIME type, size, and required-input validation rules.
HTTP file transfer	Reuse	Axios	MIT	Provides multipart HTTP requests, upload progress callbacks, cancellation, and structured error handling.
Upload progress UI	Build	React components	Custom work	Displays bytes transmitted, percentage, progress bar, upload state, and actionable errors.
Video preview	Reuse + Build	HTML5 <video> element	Browser standard + custom work	Reuses browser video playback and implements a simple preview interface. Codec support varies across browsers.

Upload session management	Build	React useState and Context API	MIT (React) + custom work	Tracks selected file, task description, upload state, and server response without an additional state-management library.
Backend API integration	Build	TypeScript API service using Axios	Custom work	Connects C1 to E4's existing /api/v1/videos endpoint and validates returned JSON metadata.
Navigation to C2	Build + Reuse	React application navigation	Custom work; depends on selected router	Passes the resulting video_id and task description to the configuration stage. A routing library may be reused if already present.
Error feedback	Build	React UI components	Custom work	Displays user-friendly messages for unsupported formats, failed requests, size violations, and upload interruptions.

API Contract

Endpoint	**Inputs**	**Outputs**	**Errors**
POST /api/v1/videos	multipart/form-data: file (Binary, required), MP4/MOV, 1 byte–2 GiB.	201: {video_id: Int, storage_key: String, size_bytes: Int, sha256: String}	400 unsupported/empty file; 409 checksum duplicate if enabled; 413 size limit exceeded; 500 storage error.
GET /api/v1/videos/{video_id}	Path: video_id (Int ≥ 1, required).	200: {video_id: Int, storage_key: String, size_bytes: Int, duration_s: Float|null}	400 invalid video ID; 404 video not found; 500 metadata error.
validateVideoFile	file: File | null	{valid: boolean, errors: string[]}	Returns validation

			failures for missing, empty, unsupported, or oversized files.
uploadVideo	file: File, onProgress: (pct: number) => void, optional AbortSignal	Promise<VideoUploadResponse>	Rejects on network, validation, cancellation, or backend errors.
getVideoMetadata	videoId: number	Promise<VideoMetadataResponse>	Rejects for invalid IDs, missing records, or server errors.
proceedToConfiguration	videoId: number, taskDetails: string	Updates shared frontend state and navigates to C2.	Blocks navigation if videoId is missing or invalid.

C2

Data Flow:



[Boxes represent data entities and frontend data structures along with their attributes; PK = primary key; FK = foreign key; Lines represent relationships between entities; Numbers represent cardinality]

Structural Decisions:

Structural Decision	Decision Rationale
Configuration = Frontend state	Stores selected training parameters, validation results, and confirmation status before training starts.
TrainingParameters = Configuration object	Separates configurable training settings from training records that were performed and simplifies validation.
TrainingJob = E4 entity	Uses existing persistent entity from D1 and stores information required to perform and monitor training.
training_parameter = String	Allows values for configuration to be stored and transmitted as a single validated parameter set.
validation_feedback = String	Records parameter validation messages returned to the user before training starts.
confirmation_status = String	Indicates whether a configuration has been accepted and is ready to be trained or reviewed.
video_id = integer	References to the uploaded video during C1.
parameter_id = integer	Provides a unique identifier for a training parameter set.
C2 -> TrainingJob = Temporary-to-persistent relationship	Configuration exists only while settings are being edited by the user. When successful, it will create or update a persistent TrainingJob record.
Data model = Frontend state + E4 relational reference	Duplication of data maintained by E4 is prevented but the relation between configuration and training records is still shown.

Core Algorithms:

Algorithm 1: Training Parameter Validation

What	Validates training parameters set by the user and checks that all required values are present before the configuration proceeds to review.
I&O	**Input**: video_id (Integer); training_parameters (String) **Output**: validation result (Boolean), validation_feedback (String)

Complexity	O(1) time and auxiliary memory when validating against a fixed set of configuration rules and acceptable parameter ranges.
Why	Prevents invalid configurations from reaching the training system and provides feedback to the user immediately.
Edge Cases	Missing parameter values, invalid parameter format, empty configuration, unsupported training mode, corrupted configuration data, or missing video_id.

Algorithm 2: Configuration and Training Job Creation

What	Creates or updates a TrainingJob record after configuration parameters are successfully validated and settings are confirmed by the user.
I&O	**Input**: Validated configuration data **Output**: TrainingJob record with training_id and status
Complexity	O(1) time and memory for one TrainingJob creation or update.
Why	Ensures only validated configurations are sent to the Review and Training components.
Edge Cases	User cancels before confirmation, database write failure, duplicate training request, invalid video_id reference, lost network connection while submitting, or TrainingJob creation timeout.

Build vs Reuse Decisions

What	**B or R**	**Library / Service**	**License**	**Reason**
Configuration form	Build	React Components	Custom work	Creates the user interface for entering and altering training parameters.
Parameter validation	Build	TypeScript functions	Custom work	Implements SkillForge-specific validation rules for training settings.
Configuration state management	Build	React useState and Context API	MIT (React) + custom work	Tracks configuration data and validation results across components.

Backend configuration API	Build	TypeScript API service using Axios	Custom work	Sends configuration data and receives validation responses.
HTTP communication	Reuse	Axios	MIT	Provides request handling and error processing.
Navigation to C3	Build + Reuse	React routing solution	Custom work	Transfers validated configuration data to the Review component.
Error feedback UI	Build	React UI components	Custom work	Displays validation failures and submission errors.
TrainingJob persistence	Reuse	E4 Storage Layer	Existing system	Stores training request information using the existing backend architecture.

API Contract

Endpoint	**Inputs**	**Outputs**	**Errors**
POST /api/v1/configuration	video_id (Integer, required), training_parameter (String, required)	201: {parameter_id: Integer, confirmation_status: String}	400 invalid parameters; 404 video not found; 500 validation error
GET /api/v1/configuration/{parameter_id}	parameter_id (Integer ≥ 1, required)	200: {parameter_id: Integer, training_parameter: String}	400 invalid ID; 404 configuration not found; 500 server error
validateConfiguration	videoId: number, training_parameter: String	{valid: boolean, validationFeedback: string}	Returns validation failures for missing or

			invalid configuration data
createTrainingJob	validatedConfiguration: Configuration	Promise	Rejects on backend, network, database, or validation errors
getConfiguration	parameterId: number	Promise	Rejects for invalid IDs, missing records, or server errors
proceedToReview	parameterId: number	Updates shared frontend state and navigates to C3	Blocks navigation if validation fails or configuration is incomplete

C3

Data Flow:

No persistent data

Core Algorithms:

Algorithm 1: Training Configuration Review and Validation

Column	Description

What	Retrieve the selected video information and training configuration from C1/C2, display them in a summary, and validate that all required information is present before allowing training to begin.
I&O	Input: video_id (Integer ≥ 1), task details (String), and training parameters (epochs: Integer 1–1000, learning_rate: Float > 0 and ≤ 1, optional seed: Integer 0–2147483647). Output: Reviewable summary containing video and configuration details, validation result (Boolean), and error messages (String array).
Complexity	O(P) time to inspect P training parameters and O(P) memory to construct validation results and a review summary. With the current fixed set of parameters, both are effectively O(1).
Why	Ensures that the user has an opportunity to verify training settings before initiating potentially expensive reconstruction, simulation, and ML training processes. Detecting invalid inputs prevents unnecessary backend requests.
Edge Cases	Missing video reference, invalid training parameter values, incomplete configuration, unsupported parameter combinations, inaccessible video records, or stale frontend state following page refresh. Display validation errors and prevent submission until required fields are valid.

Build vs Reuse Decisions

What	**B or R**	**Library / Service**	**License**	**Reason**
Review summary interface	Build	React components	Custom work	Presents the selected demonstration and training parameters in a readable format before submission.
Configuration validation	Build	TypeScript functions	Custom work	Implements project-specific checks for missing video references and invalid training settings.
Navigation between components	Reuse + Build	Existing React routing solution	Depends on selected library	Supports returning to C2 for corrections and proceeding to C4 after successfully creating a training job.

Training request construction	Build	TypeScript API service	Custom work	Converts reviewed configuration data into the payload expected by E4's existing training-job endpoint.
Submission status handling	Build	React components	Custom work	Prevents duplicate requests, displays loading indicators, and communicates success or failure.
Backend training job persistence	Reuse	E4 FastAPI + SQLite API	Existing backend implementation	Uses the training-job endpoints and relational data model already defined in D1 rather than introducing additional persistence for C3.

API Contract

Endpoint	**Inputs**	**Outputs**	**Errors**
GET /api/v1/videos/{video_id}	Path: video_id (Integer ≥ 1).	200: {video_id: Int, storage_key: String, size_bytes: Int, duration_s: Float|null}	400 invalid ID; 404 video missing; 500 metadata error.
POST /api/v1/training-jobs	JSON: video_id (Integer ≥ 1), parameters (Object, required): epochs (Integer 1–1000), learning_rate (Float > 0 and ≤ 1),	201: {training_id: Int, video_id: Int, status: "queued", progress_pct: 0}	404 video missing; 422 invalid parameters; 500 database error.

	optional seed (Integer 0–2147483647).		
GET /api/v1/training-jobs/{training_id}	Path: training_id (Integer ≥ 1).	200: {training_id: Int, video_id: Int, parameters: Object, status: String, progress_pct: Float}	400 invalid ID; 404 training job missing; 500 database error.
validateReviewConfiguration	videoId: number, parameters: TrainingConfig	{valid: boolean, errors: string[]}	Returns missing or invalid parameter messages.
getReviewVideoMetadata	videoId: number	Promise<VideoMetadataResponse>	Rejects if the video is missing or the backend request fails.
returnToConfiguration	Current TrainingConfig and videoId	Preserves settings and navigates to C2.	Prevents losing existing configuration state.
createTrainingJob	videoId: number, parameters: TrainingConfig	Promise<TrainingJobResponse>	Rejects on backend validation, network, or database errors.

C4

Data Flow: No Persistent Data

Core Algorithms:

Algorithm 1: Status Report Calculation

What	Determines a status on the training model by examining the data retrieved from it.
I & O	Input: Model_id (int) Outputs: Status (String)
Complexity	O(n) to retrieve all information from the data and then enumerate a string based on those values.
Why	It's important to have a monitor for a model while it undergoes training so that any errors and mistakes can be corrected as soon as possible.
Edge Cases	Model does not exist; values retrieved from model are unexpected / negative.

Build vs Reuse Decisions

What	B or R	Library / Service	License	Reason
Frontend visual components	Build	React - custom	None	This will be the area users spend the most time with outside of the configuration. It needs to clearly show what is going on and allow users insight into how the product is working.

API Contract

Proposed REST API: /api/v1.

Endpoint	Inputs	Outputs	Errors
GET /api/v1/monitor/{status}	model_id: path int >= 1. required	200: {status: string}	400: invalid model id

POST /api/v1/monitor	Final_report: JSON	201: {status: string}	500: Database error

3. Edge Cases and Error Handling

	Boundary Values	Timing and Concurrency	Bad or Missing Input	External Dependencies
E1	Expired/invalid state or cookie -> E1 rejects, re-login.	Simultaneous logout/request -> E1 validates active session.	Missing callback code -> E1 returns 400.	OIDC provider unavailable -> E1 returns 503; retry.
E3	3D Mesh file is too big (>100 GB)	Multiple requests to train a model while the component is in a busy training state.	3D Mesh File is invalid/corrupted. Model of mesh file already exists	Database unavailable when obtaining mesh file or storing completed physic/diffusion models.
E4	Empty/>2 GiB file -> E4 400/413.	Concurrent writes -> E4 transactions and unique IDs.	Unknown video ID/format -> E4 404/400.	Disk or database unavailable -> E4 500 + cleanup.
C1	File too large (>10 GB)	Multiple users attempt to stream high resolution videos at once	Inaccessible Local file	Internet infrastructure us down
C2	Parameter value exceeds allowed range; empty configuration; invalid video_id	User modifies settings rapidly while validation is running; multiple configuration requests	Missing training parameter; missing video_id; invalid parameter format;	Backend API unavailable; database unavailable when creating TrainingJob; network failure

		submitted simultaneously	incomplete configuration	during configuration submission
C4	Status report is too large to store on database (> 1 GB)	Multiple agents request a status on the same training model and overload the monitor.	Request confirmation is empty/Model does not exist.	Database is unavailable, training status cannot be obtained.

4. Requirements Traceability Table

ID	Description	Priority	Satisfying Design Element	How Component Satisfies
REQ-1	Provide pathway for developing advanced robot capabilities	High - Covered	E3 /api/v1/diffusion_model/{model_id}	Handles training a diffusion model on prepared robot actions to develop a neural-network powered robot skill.
REQ-2	Automatically extract relevant actions from demonstration videos	High - Covered	E2 Action Extraction Pipeline	Processes uploaded demonstration videos and automatically identifies relevant actions, reducing the need for manual labeling.
REQ-3	Provides security to prevent unauthorized users from accessing data and models that they have no permission to look at.	High - Covered	E1 Account Authentication	Authenticates browser requests and only permits users that have been granted access by the server to view and create models.
REQ-4	Generate training data from 3rd-person perceptive video	Medium - Covered	E2 - Action Extraction Pipeline (extracts specifically from 3rd person video)	Allows the construction of robot action data from videos of humans

				performing tasks. This is done through the VGGT->3DGS pipeline which can then be processed with segmentation and pose-estimation to develop human positions at each time-step. Reverse-kinematics can then reconstruct a set of required robot actions.
REQ-5	Train from videos with minimal camera quality/viewpoint requirements	High - Partially covered; tests pending	C1 POST /api/v1/videos; E4 VideoAsset.file_format, duration; E3 VGGT reconstruction flow (D0 data-flow figure); E4-01/E4-03	C1/E4 accept common MP4/MOV demonstrations and persist source metadata. E3 reconstruction may support varied views, but minimum viable camera/viewpoint thresholds must be tested and documented.

5. Design Rational

Choice Name	Choice Description	Other Info
3D Reconstruction Method	We chose to use VGGT as our 3D reconstruction method because we want to be able to replicate scenes consisting of high motion and many actors, and VGGT provides the features necessary to handle dynamic scenes which typical gaussian splatting is not able to handle off the start.	
3D Reconstruction Representation	We chose to use Diffusion Models as our 3D reconstruction representation because they can be used by robotic agents to replicate actions by generating the scene along with any simulation data we provide.	
Parameter Selection	We chose a configurable parameter-based approach over using fixed training settings because certain demonstrations	Implemented through C2 Configuration component.

	may need different levels of processing and model complexity. We accepted the trade-off of additional validation complexity because it is important to have variety and flexibility for tasks.	
Frontend-Stack	We chose React and TypeScript over JavaScript because it provides reusable components, better maintainability, and checks typing. We accepted the trade-off of a larger initial setup cost because long-term development productivity and code quality are improved.	Supports C1 Upload, C2 Configuration, and UI components.
Database	We propose SQLite over PostgreSQL because metadata-only relational workloads are small and an embedded DB simplifies the Python prototype. We accept limited multi-writer concurrency because the initial deployment is expected to be a single server.	Proposed; not formally confirmed. Binary media lives on disk. D0 I15/I16 FTPS vs D2 REST upload should be reconciled with E2. Reference D0 pipeline and D1 E4 schema.
Backend Web Framework	We propose Python/FastAPI over Flask because typed request validation and generated API contracts help integrate the training pipeline. We accept extra dependency and framework-learning overhead for clearer C1/E2/E4 interfaces.	Proposed; not formally confirmed. Authentication E1 would reuse OIDC rather than custom passwords. Align with D0 I11/I12 and API paths before approval.
3D Mesh Development	We chose the DNLR foundation model over basic TSDF fusion algorithm because DNLR offers higher visual fidelity. We accepted the trade-off of slower computation because high-quality meshes are 100% required in our case."	
Physics Engine	We chose MuJoCo over NVIDIA Issac Sim because MuJoCo does not limit processing to NVIDIA GPUs and is generally viewed as far less computationally intensive. We accepted the trade-off of potentially lower visual fidelity because the high compute cost of Issac Sim was a no-go for our project, and because similar levels of visual fidelity may be achieved in other ways."	

6. Peer Design Review Notes

- Did not do activity in class (no activity was available to do)